

Module Title: Reasoning about Programs

Module Code: 6SENG001W, 6SENG003C

Exam Period: January 2020

Time Allowed: 2 Hours

INSTRUCTIONS FOR CANDIDATES

PLEASE WRITE YOUR STUDENT ID CLEARLY AT THE TOP OF EACH PAGE.

You are advised (but not required) to spend the first ten minutes of the examination reading the questions and planning how you will answer those you have selected.

Answer ALL questions in Section A and TWO questions from Section B.

Section A is worth a total of 50 marks.

Each question in section B is worth 25 marks.

In section B, only the TWO questions with the HIGHEST MARKS will count towards the FINAL MARK for the EXAM.

The B-Method's Abstract Machine Notation (AMN) is given in Appendix B.

**THIS PAPER MUST NOT BE TAKEN OUT OF THE EXAMINATION ROOM
DO NOT TURN OVER THIS PAGE UNTIL THE INVIGILATOR INSTRUCTS YOU TO DO SO**

Section A

Answer ALL questions from this section.

You may also wish to consult the B-Method notation given in Appendix B.

Question 1

You are given the following collection of B set and function declarations for the Scottish Islands:

$$\text{Scottish_Islands} = \{ \text{Skye}, \text{Islay}, \text{Mull}, \text{Jura}, \text{Lewis_and_Harris}, \\ \text{North_Uist}, \text{South_Uist}, \text{Benbecula} \}$$

$$\text{Inner_Hebrides} \in \mathbb{P}(\text{Scottish_Islands})$$

$$\text{Inner_Hebrides} = \{ \text{Skye}, \text{Islay}, \text{Mull}, \text{Jura} \}$$

$$\text{Outer_Hebrides} \in \mathbb{P}(\text{Scottish_Islands})$$

$$\text{Outer_Hebrides} = \{ \text{Lewis_and_Harris}, \text{North_Uist}, \text{South_Uist}, \text{Benbecula} \}$$

$$\text{highest_point} \in \text{Scottish_Islands} \rightarrow \mathbb{N}$$

$$\text{highest_point} = \{ \text{Skye} \mapsto 993, \text{Islay} \mapsto 491, \text{Mull} \mapsto 966, \text{Jura} \mapsto 785, \\ \text{Lewis_and_Harris} \mapsto 799, \text{North_Uist} \mapsto 347, \\ \text{South_Uist} \mapsto 620, \text{Benbecula} \mapsto 124 \}$$

Evaluate the following expressions:

- | | |
|--|-------------------|
| (a) $\text{Outer_Hebrides} \cap \{ \text{Skye}, \text{South_Uist}, \text{Mull}, \text{Benbecula} \}$ | [1 mark] |
| (b) $\text{Inner_Hebrides} - \{ \text{North_Uist}, \text{Islay}, \text{Jura} \}$ | [2 marks] |
| (c) $\text{card}(\text{highest_point})$ | [1 mark] |
| (d) $\text{Scottish_Islands} \cap \text{dom}(\text{highest_point})$ | [1 mark] |
| (e) $\text{ran}(\text{highest_point})$ | [1 mark] |
| (f) $\text{highest_point}(\text{Lewis_and_Harris})$ | [1 mark] |
| (g) $\text{Inner_Hebrides} \triangleleft \text{highest_point}$ | [2 marks] |
| (h) $\text{highest_point} \triangleright 0..900$ | [3 marks] |
| (i) $\mathbb{P}(\{ \text{Skye}, \text{Mull}, \text{Jura} \})$ | [3 marks] |
| | [TOTAL 15] |

Question 2

Given the two relations:

$$R \in \text{LETTER} \leftrightarrow \mathbb{N}$$

$$Q \in \mathbb{N} \leftrightarrow \text{COLOUR}$$

$$R = \{ (a, 1), (a, 2), (b, 2), (c, 3), (d, 4), (e, 5) \}$$

$$Q = \{ (1, \text{red}), (2, \text{blue}), (3, \text{green}), (5, \text{purple}) \}$$

- (a) The two relations R and Q *could be* composed using the relational composition operator “;” in two ways:

$$R ; Q$$

$$Q ; R$$

Explain why one of these compositions is possible and why one is not possible.

[2 marks]

- (b) Evaluate the “possible” composition of R and Q .

[3 marks]

- (c) For each of the relations R and Q state whether it is just a relation or is also a function. In addition, give the justification for your decisions.

[2 marks]

[TOTAL 7]

Question 3

- (a) The building block of a B-method specification is the concept of an *Abstract Machine (AM)*. Explain what a B Abstract Machine is, in particular, you should answer the following questions:

- What is it a specification of?
- What are its main logical parts and what are the relationships between the parts?
- What is it similar to in terms of a programming language feature?
- Are there any logical components of a B machine that are not represented in your chosen programming language feature? What does this mean for the programming language feature?

[10 marks]

- (b) Describe the three categories of states that a B machine can be in and illustrate your answer by means of a diagram. What clause of a B machine determines which state is which?

[8 marks]

[TOTAL 18]

Question 4

- (a) Explain in your own words the meaning of the Hoare triple

$$[y > z + 1] \ x := z \ [x < y]$$

[2 marks]

- (b) Which of the following Hoare triples are valid? Give a counterexample for each invalid triple.

(i) $[x < y] \ x := y \ [false]$

[2 marks]

(ii) $[x < y] \ x := y \ [true]$

[2 marks]

(iii) $[x < y] \ y := y + 1 \ [x < y + 1]$

[2 marks]

(iv) $[x = 3] \ x := y \ [y = 3]$

[2 marks]

[TOTAL 10]

Section B

Answer TWO questions from this section.

You may wish to consult the B-Method notation given in Appendix B.

Question 5

Write a B-Method machine that specifies a *queue* of customers waiting to be served at a small village Post Office counter. Due to the Post Offices small size, the queue has to have a maximum length for customers.

Your B machine should deal with error handling where required and should include the following:

- (a) Any sets, constants and variables, and any state invariant that the Post Office's customer queue requires. [9 marks]
 - (b) The Post Office queue operations:
 - (i) *JoinPOQueue* – a new customer joins the end of the queue. [7 marks]
 - (ii) *GotoCounter* – the next customer leaves the front of the queue and goes to the counter to be served. [6 marks]
 - (iii) *CustomersWaiting* – reports via a suitable message whether the queue is empty then no; otherwise yes. [3 marks]
- [TOTAL 25]**

Question 6

Appendix A contains the Club B abstract machine, this specifies a simple club's membership list scheme.

The club has a number of members. To join the club a new member must first join the waiting list then he/she can then become a club member.

The system provides the following operations:

- Joining the club as a member.
- Joining the club's waiting list to become a member.
- Leaving the club.
- Reset the club's membership and waiting lists.
- Checking if someone is a member of the club.

With reference to the Club machine, *using "plain English"* answer the following questions.

- (a) Give the meaning of the six predicates that form the Club's invariant, as given in the INVARIANT clause:

```
20      queuetotal < capacity  &
21      members <: NAME &
22      waiting <: NAME &
23      members /\ waiting = {} &
24      card(members) <= capacity &
25      card(waiting) <= queuetotal
```

[12 marks]

- (b) Explain the meaning of the *preconditions* for the operations:

(i) join

[2 marks]

(ii) join_queue

[4 marks]

(iii) remove

[2 marks]

- (c) Draw the *Structure Diagram* for the Club machine.

[5 marks]

[TOTAL 25]

Question 7

- (a) Find the missing assertions using pre-condition propagation.

```
[assertion 1]
  y:=y-1;
[assertion 2]
  x:=x+z;
[assertion 3]
  y:=x+y
[x=y+1]
```

[9 marks]

- (b) Find suitable intermediate assertions for the following Hoare triple; this involves finding an invariant for the loop.

```
[u>0]
x:=0;
y:=0;
[invariant]
WHILE y<v DO
[assertion 1]
  x:=x+u;
[assertion 2]
  y:=y+1
[assertion 3]
END
[x=u*v]
```

[16 marks]

[TOTAL 25]

Appendix A. Club B Machine

The following is a B Machine – Club that specifies a club's membership scheme.

```
1  MACHINE Club(NAME, capacity)
2
3  CONSTRAINTS
4      capacity : NAT1 & 5 <= capacity &
5      capacity < card(NAME)
6
7  SETS
8      ANSWER = { yes, no }
9
10  CONSTANTS
11      queuetotal
12
13  PROPERTIES
14      queuetotal : NAT1 & queuetotal > 2
15
16  VARIABLES
17      members, waiting
18
19  INVARIANT
20      queuetotal < capacity &
21      members <: NAME &
22      waiting <: NAME &
23      members /\ waiting = {} &
24      card(members) <= capacity &
25      card(waiting) <= queuetotal
26
27  INITIALISATION
28      members := {} || waiting := {}
29
```

[Continued on next page.]


```
30  OPERATIONS
31
32  join( newmember ) =
33      PRE newmember : waiting & card(members) < capacity
34      THEN  members := members \/ { newmember }
35          || waiting := waiting - { newmember }
36      END ;
37
38  join_queue( newmember ) =
39      PRE newmember : NAME & newmember /: members &
40          newmember /: waiting & card(waiting) < queuetotal
41      THEN waiting := waiting \/ { newmember }
42      END ;
43
44  remove( member ) =
45      PRE member : members
46      THEN members := members - { member }
47      END ;
48
49  reset_club_lists
50      BEGIN
51          members, waiting := {}, members
52      END ;
53
54  ans <-- is_member( member ) =
55      PRE member : NAME
56      THEN
57          IF ( member : members )
58              THEN ans := yes
59              ELSE ans := no
60          END
61      END
62
63  END /* Club */
```

Appendix B. B-Method's Abstract Machine Notation (AMN)

The following tables present AMN in two versions: the “pretty printed” symbol version & the ASCII machine readable version used by the B tools: *Atelier B* and *ProB*.

B.1 AMN: Number Types & Operators

B Symbol	ASCII	Description
$\mathbb{N}$	NAT	Set of natural numbers from 0
$\mathbb{N}_1$	NAT1	Set of natural numbers from 1
$\mathbb{Z}$	INTEGER	Set of integers
$\text{pred}(x)$	pred(x)	predecessor of x
$\text{succ}(x)$	succ(x)	successor of x
$x + y$	x + y	x plus y
$x - y$	x - y	x minus y
$x * y$	x * y	x multiply y
$x \div y$	x div y	x divided by y
$x \bmod y$	x mod y	remainder after x divided by y
x^y	x ** y	x to the power y , x^y
$\min(A)$	min(A)	minimum number in set A
$\max(A)$	max(A)	maximum number in set A
$x .. y$	x .. y	range of numbers from x to y inclusive

B.2 AMN: Number Relations

B Symbol	ASCII	Description
$x = y$	x = y	x equal to y
$x \neq y$	x /= y	x not equal to y
$x < y$	x < y	x less than y
$x \leq y$	x <= y	x less than or equal to y
$x > y$	x > y	x greater than y
$x \geq y$	x >= y	x greater than or equal to y

B.3 AMN: Set Definitions

B Symbol	ASCII	Description
$x \in A$	$x : A$	x is an element of set A
$x \notin A$	$x /: A$	x is not an element of set A
$\emptyset, \{ \}$	$\{ \}$	Empty set
$\{ 1 \}$	$\{ 1 \}$	Singleton set (1 element)
$\{ 1, 2, 3 \}$	$\{ 1, 2, 3 \}$	Set of elements: 1, 2, 3
$x .. y$	$x .. y$	Range of integers from x to y inclusive
$\mathbb{P}(A)$	$\text{POW}(A)$	Power set of A
$\text{card}(A)$	$\text{card}(A)$	Cardinality, number of elements in set A

B.4 AMN: Set Operators & Relations

B Symbol	ASCII	Description
$A \cup B$	$A \ \backslash / \ B$	Union of A and B
$A \cap B$	$A \ /\ \ B$	Intersection of A and B
$A - B$	$A - B$	Set subtraction of A and B
$\bigcup AA$	$\text{union}(AA)$	Generalised union of set of sets AA
$\bigcap AA$	$\text{inter}(AA)$	Generalised intersection of set of sets AA
$A \subseteq B$	$A <: B$	A is a subset of or equal to B
$A \not\subseteq B$	$A /<: B$	A is not a subset of or equal to B
$A \subset B$	$A <<: B$	A is a strict subset of B
$A \not\subset B$	$A /<<: B$	A is not a strict subset of B
$\{ x \mid x \in TS \wedge C \}$	$\{ x \mid x : TS \ \& \ C \}$	Set comprehension

B.5 AMN: Logic

B Symbol	ASCII	Description
$\neg P$	not P	Logical negation (not) of P
$P \wedge Q$	P & Q	Logical and of P, Q
$P \vee Q$	P or Q	Logical or of P, Q
$P \Rightarrow Q$	P => Q	Logical implication of P, Q
$P \Leftrightarrow Q$	P <=> Q	Logical equivalence of P, Q
$\forall xx \cdot (P \Rightarrow Q)$	!(xx).(P => Q)	Universal quantification of xx over $(P \Rightarrow Q)$
$\exists xx \cdot (P \wedge Q)$	#(xx).(P & Q)	Existential quantification of xx over $(P \wedge Q)$
<i>TRUE</i>	TRUE	Truth value <i>TRUE</i> .
<i>FALSE</i>	FALSE	Truth value <i>FALSE</i>
<i>BOOL</i>	BOOL	Set of boolean values $\{ \textit{TRUE}, \textit{FALSE} \}$
<i>bool</i> (P)	bool(P)	Convert predicate P into <i>BOOL</i> value

B.6 AMN: Ordered Pairs & Relations

B Symbol	ASCII	Description
$X \times Y$	X * Y	Cartesian product of X and Y
$x \mapsto y$	x -> y	Ordered pair, maplet
$\text{prj}_1(S, T)(x \mapsto y)$	prj1(S,T)(x -> y)	Ordered pair projection function
$\text{prj}_2(S, T)(x \mapsto y)$	prj2(S,T)(x -> y)	Ordered pair projection function
$\mathbb{P}(X \times Y)$	POW(X * Y)	Set of relations between X and Y
$X \leftrightarrow Y$	X <-> Y	Set of relations between X and Y
$\text{dom}(R)$	dom(R)	Domain of relation R
$\text{ran}(R)$	ran(R)	Range of relation R

B.7 AMN: Relations Operators

B Symbol	ASCII	Description
$A \triangleleft R$	A < R	Domain restriction of R to the set A
$A \triangleleft R$	A << R	Domain subtraction of R by the set A
$R \triangleright B$	R > B	Range restriction of R to the set B
$R \triangleright B$	R >> B	Range anti-restriction of R by the set B
$R[B]$	R[B]	Relational Image of the set B of relation R
$R_1 \triangleleft R_2$	R1 <+ R2	R_1 overridden by relation R_2
$R ; Q$	(R ; Q)	Forward Relational composition
$\text{id}(X)$	id(X)	Identity relation
R^{-1}	R~	Inverse relation
R^n	iterate(R,n)	Iterated Composition of R
R^+	closure1(R)	Transitive closure of R
R^*	closure(R)	Reflexive-transitive closure of R

B.8 AMN: Functions

B Symbol	ASCII	Description
$X \rightarrowtail Y$	X +-> Y	Partial function from X to Y
$X \rightarrow Y$	X --> Y	Total function from X to Y
$X \rightarrowtail Y$	X >+> Y	Partial injection from X to Y
$X \rightarrowtail Y$	X >-> Y	Total injection from X to Y
$X \twoheadrightarrowtail Y$	X +->> Y	Partial surjection from X to Y
$X \twoheadrightarrow Y$	X -->> Y	Total surjection from X to Y
$X \twoheadrightarrowtail Y$	X >->> Y	(Total) Bijection from X to Y
$f \triangleleft g$	f <+ g	Function f overridden by function g

B.9 AMN: Sequences

B Symbol	ASCII	Description
$[]$	<code>[]</code>	Empty Sequence
$[e1]$	<code>[e1]</code>	Singleton Sequence
$[e1, e2]$	<code>[e1, e2]</code>	Constructed (enumerated) Sequence
$\text{seq}(X)$	<code>seq(X)</code>	Set of Sequences over set X
$\text{iseq}(X)$	<code>iseq(X)</code>	Set of injective Sequences over set X
$\text{size}(s)$	<code>size(s)</code>	Size (length) of Sequence s

B.10 AMN: Sequences Operators

B Symbol	ASCII	Description
$s \frown t$	<code>s^t</code>	Concatenation of Sequences s & t
$e \rightarrow s$	<code>e -> s</code>	Insert element e to front of sequence s
$s \leftarrow e$	<code>s <- e</code>	Append element e to end of sequence s
$\text{rev}(s)$	<code>rev(s)</code>	Reverse of sequence s
$\text{first}(s)$	<code>first(s)</code>	First element of sequence s
$\text{last}(s)$	<code>last(s)</code>	Last element of sequence s
$\text{front}(s)$	<code>front(s)</code>	Front of sequence s , excluding last element
$\text{tail}(s)$	<code>tail(s)</code>	Tail of sequence s , excluding first element
$\text{conc}(SS)$	<code>conc(SS)</code>	Concatenation of sequence of sequences SS
$s \uparrow n$	<code>s /\ n</code>	Take first n elements of sequence s
$s \downarrow n$	<code>s \\/ n</code>	Drop first n elements of sequence s

B.11 AMN: Miscellaneous Symbols & Operators

B Symbol	ASCII	Description
$\text{var} := E$	<code>var := E</code>	Assignment
$S1 \parallel S2$	<code>S1 S2</code>	Parallel execution of $S1$ and $S2$

B.12 AMN: Operation Statements

B.12.1 Assignment Statements

`xx := xxval`

`xx, yy, zz := xxval, yyval, zzval`

`xx := xxval || yy := yyval`

B.12.2 Deterministic Statements

`skip`

`BEGIN S END`

`PRE PC THEN S END`

`IF B THEN S END`

`IF B THEN S1 ELSE S2 END`

`IF B1 THEN S1 ELSIF B2 THEN S2 ELSE S3 END`

`CASE E OF`

`EITHER v1 THEN S1`

`OR v2 THEN S2`

`OR v3 THEN S3`

`ELSE`

`S4`

`END`

B.13 B Machine Clauses

MACHINE Name(Params)

CONSTRAINTS	Cons
EXTENDS	M1, M2, ...
INCLUDES	M3, M4, ...
PROMOTES	op1, op2, ...
SEES	M5, M6, ...
USES	M7, M8, ...
SETS	Sets
CONSTANTS	Consts
PROPERTIES	Props
VARIABLES	Vars
INVARIANT	Inv
INITIALISATION	Init

OPERATIONS

```

yy <-- op( xx ) =
    PRE PC
    THEN Subst
    END ;
...
END
```